Musterlösung zur Übung 5

Methoden

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Hinweis: Die Lösungen zeigen einen möglichen Weg zu allen zehn Aufgaben. Andere Programme sind ebenfalls richtig, wenn sie die Vorgaben erfüllen. Es gelten die Eingabevoraussetzungen des Übungsblatts.

Java: Verwenden Sie Java 25 oder höher und je Aufgabe ein Compact File im Projekt `Prog1`. Eigene Methoden stehen neben `void main()`. Ergänzungen für `main` sind ausdrücklich gekennzeichnet. Die beiden Varianten von Aufgabe 9 stehen in getrennten Dateien.

01 Methoden als Blackbox entwerfen | Basic

a–b) Mögliche Methodenköpfe und Aufrufe:

Auftrag	Methodenkopf und Beispielaufruf
Festes Menü anzeigen	`void zeigeMenue()` `zeigeMenue();`

Quader mit drei Seitenlängen	`double volumen(double laenge,`
	`  double breite, double hoehe)`
	`double v = volumen(2, 3, 4);`

Text mehrfach anzeigen	`void wiederhole(String text, int anzahl)` `wiederhole("Hallo", 3);`
Ganze Zahl einlesen	`int liesZahl()` `int n = liesZahl();`

Das feste Menü benötigt keine Information vom Aufrufer. Für das Volumen fehlen die drei Seitenlängen; das Ergebnis wird als `double` zurückgegeben. Die wiederholte Ausgabe braucht Text und Anzahl, liefert aber keinen Wert. `liesZahl` erhält seine Information von der Tastatur und gibt sie als `int` zurück. Menü- und Textausgabe haben den Rückgabetyp `void`.

c) Vollständiges Beispiel. Die beiden Volumina betragen 24 und 9:

```
void main() {
    zeigeMenue();
    IO.println(volumen(2, 3, 4));   // 24.0
    IO.println(volumen(1.5, 2, 3)); // 9.0
}

double volumen(double laenge, double breite, double hoehe) {
    return laenge * breite * hoehe;
}

void zeigeMenue() {
    IO.println("1: Volumen berechnen");
    IO.println("0: Beenden");
}
```

02 Rückgabe und Bildschirmausgabe unterscheiden | Basic

a) Die Ausgaben lauten `4` und `10`. `verdopple(a)` berechnet zwar 8, doch der Aufrufer verwendet diesen Rückgabewert nicht. Die Variable `a` bleibt 4, weil die Methode nur eine Kopie ihres Werts erhält. Beim zweiten Aufruf wird zuerst `a + 1`, also 5, berechnet; dessen Doppeltes wird ausgegeben.

b) `zeigeDoppeltes` hat den Rückgabetyp `void`. Eine Bildschirmausgabe stellt keinen Wert für eine Zuweisung bereit. Deshalb ist `int b = zeigeDoppeltes(3);` ungültig. Richtig ist:

```
int b = verdopple(3); // b ist 6
```

c) `return;` beendet nur den aktuellen Aufruf von `zeigeDoppeltes`. `main` setzt danach mit der nächsten Anweisung fort. Zusammengesetztes Testprogramm für a) bis c):

```
void main() {
    int a = 4;
    verdopple(a);
    IO.println(a);                    // 4
    IO.println(verdopple(a + 1));     // 10
    int b = verdopple(3);
    IO.println(b);                    // 6
    IO.println("Vor dem Aufruf");
    zeigeDoppeltes(-3);
    IO.println("Nach dem Aufruf");
}

int verdopple(int zahl) {
    return 2 * zahl;
}

void zeigeDoppeltes(int zahl) {
    if (zahl < 0) {
        return;
    }
    IO.println(2 * zahl);
}
```

Nach den Zahlen `4`, `10` und `6` erscheinen die beiden Texte direkt nacheinander:

```
Vor dem Aufruf
Nach dem Aufruf
```

`zeigeDoppeltes(-3);` erzeugt keine Ausgabe, `zeigeDoppeltes(3);` dagegen 6. `return 6;` liefert einen Wert an den Aufrufer. Mit `return;` endet eine `void`-Methode ohne Ergebnis.

Basic: Wiederverwendung und Dokumentation

03 Code auslagern und dokumentieren | Basic

a–b) Mit **Extract Method** entsteht `berechneFlaeche`; beide Berechnungen werden durch Aufrufe ersetzt. **Rename** ändert den Namen überall in `rechteckFlaeche`.

DRY – Don't Repeat Yourself: Die Berechnungsregel hat nur noch eine Stelle im Programm. Eine spätere Änderung erfolgt dort und wirkt für alle Aufrufe. Die Ausgaben bleiben in `main`, damit sich die Berechnung auch ohne Bildschirmausgabe verwenden lässt.

c) Methodendeklaration mit Doc-Comment:

```
/**
 * Berechnet die Flaeche eines Rechtecks in Quadratzentimetern.
 * @param breite Breite in Zentimetern, von 1 bis 1000
 * @param hoehe Hoehe in Zentimetern, von 1 bis 1000
 * @return Flaeche in Quadratzentimetern
 */
int rechteckFlaeche(int breite, int hoehe) {
    return breite * hoehe;
}
```

d) Passende `main` mit den ursprünglichen Berechnungen und den drei zusätzlichen Aufrufformen:

```
void main() {
    int seiteA = 3;
    int seiteB = 4;
    IO.println(rechteckFlaeche(seiteA, seiteB)); // 12
    seiteA = 5;
    seiteB = 2;
    IO.println(rechteckFlaeche(seiteA, seiteB)); // 10

    IO.println(rechteckFlaeche(6, 7));          // 42
    int eingabeA = Integer.parseInt(IO.readln("Seite A: "));
    int eingabeB = Integer.parseInt(IO.readln("Seite B: "));
    IO.println(rechteckFlaeche(eingabeA, eingabeB));
    IO.println(rechteckFlaeche(seiteA + 1, seiteB)); // 12
}
```

Beim Aufruf `rechteckFlaeche(seiteA, seiteB)` wird der Wert des ersten Arguments an `breite` und der des zweiten an `hoehe` übergeben. Entscheidend sind Reihenfolge und passende Datentypen. Die Namen der Variablen in `main` müssen nicht den Parameternamen entsprechen.

Jeder Parameter ist eine lokale Variable des neuen Methodenaufrufs. Im letzten Aufruf wird zuerst `seiteA + 1` zu 6 ausgewertet. Die Methode erhält somit 6 und 2 und liefert 12. Für die Eingaben 3 und 4 liefert auch der Aufruf mit eingelesenen Werten 12.

Basic: Gleicher Name mit verschiedenen Parameterlisten

04 Methoden überladen | Basic

a) Die drei Deklarationen stehen gemeinsam neben `main`. Die Variante mit drei Argumenten nutzt zweimal die Variante mit zwei Argumenten:

```
int minimum(int a, int b) {
    if (a < b) {
        return a;
    }
    return b;
}

int minimum(int a, int b, int c) {
    return minimum(minimum(a, b), c);
}

double minimum(double a, double b) {
    if (a < b) {
        return a;
    }
    return b;
}
```

b) Ergebnisse und gewählte Überladungen:

Aufruf	Parametertypen	Ergebnis und Typ
`minimum(3, 7)`	`int, int`	`3` als `int`
`minimum(3, 7, 5)`	`int, int, int`	`3` als `int`
`minimum(3.5, 7.2)`	`double, double`	`3.5` als `double`
`minimum(3, 7.2)`	`double, double`	`3.0` als `double`
`minimum(-5, -3)`	`int, int`	`-5` als `int`
`minimum(2, 2, 2)`	`int, int, int`	`2` als `int`
`minimum(-2.5, -2.5)`	`double, double`	`-2.5` als `double`

Bei `minimum(3, 7.2)` wird die 3 implizit nach `double` erweitert. Die Variante mit zwei `int`-Parametern passt nicht, weil 7.2 nicht implizit nach `int` umgewandelt wird.

c) Bei diesen Methoden besteht die Signatur aus dem Namen und den Parametertypen in ihrer Reihenfolge. Rückgabetyp und Parameternamen unterscheiden keine Überladungen. Daher wären sowohl `double minimum(int a, int b)` als auch `int minimum(int x, int y)` zusätzlich zur vorhandenen Variante unzulässig.

Combined: Werte kopieren und Referenzen verfolgen

05 Wertübergabe im Debugger verfolgen | Combined

a) Die vier Ausgaben lauten in dieser Reihenfolge:

```
99
Hallo!
5
Hallo
```

Beim Aufruf wird der Zahlenwert 5 nach `wert` kopiert. Bei `String` wird die Referenz auf den Text kopiert. `text` und `nachricht` verweisen anfangs auf denselben Text. Die späteren Zuweisungen verändern nur die lokalen Parameter von `aendere`. Java verwendet auch bei Referenzen Call-by-Value.

b) Die Pfeile zeigen, auf welchen Text die jeweilige Variable verweist. Beide Pfeile zum Text `"Hallo"` meinen zunächst dieselbe Speicherstelle.

Direkt nach Eintritt in `aendere`

```
main:     zahl = 5       text ------+
                                    |--> "Hallo"
aendere:  wert = 5  nachricht ------+
```

Nach den beiden Zuweisungen in `aendere`

```
main:     zahl = 5       text ----------> "Hallo"
aendere:  wert = 99 nachricht ----------> "Hallo!"
```

Durch `nachricht + "!"` entsteht ein neuer Text. Nur `nachricht` erhält die neue Referenz. Der bisherige Text wird nicht verändert.

Nach der Rückkehr zu `main`

```
main:     zahl = 5       text ----------> "Hallo"
```

Der Aufruf von `aendere` ist beendet. Seine Parameter gehören nicht mehr zum aktiven Laufzeitstack. Die Variablen in `main` sind unverändert.

c) Die Rückgabe muss der Variable in `main` zugewiesen werden:

```
void main() {
    String text = "Hallo";
    text = erweitere(text);
    IO.println(text); // Hallo!
}

String erweitere(String text) {
    return text + "!";
}
```

Ein bloßes `erweitere(text);` würde den neuen Text zwar berechnen, dessen Referenz aber ungenutzt lassen. Erst `text = erweitere(text);` ändert, auf welchen Text die Variable in `main` verweist.

Combined: Ein Treffer beendet die Suche

06 Eine Suche nicht zu früh beenden | Combined

a) Die fehlerhafte Methode liefert bereits beim ersten geprüften Zeichen ein Ergebnis:

Aufruf	Erwartet	Fehlerhafter Code
`enthaelt("Methoden", 'h')`	`true`	`false`
`enthaelt("Methoden", 'M')`	`true`	`true`
`enthaelt("Methoden", 'x')`	`false`	`false`
`enthaelt("", 'h')`	`false`	`false`

Im Debugger zeigt der erste Fall: `i` ist 0, `text.charAt(i)` ist `'M'`. Der Vergleich mit `'h'` ist falsch. Das `else` führt sofort `return false;` aus. Der Aufruf endet, bevor `i++` ausgeführt wird. Das gesuchte `'h'` an Index 3 wird deshalb nie erreicht.

b) Die Rückgabe für „nicht gefunden“ gehört hinter die Schleife:

```
boolean enthaelt(String text, char gesucht) {
    for (int i = 0; i < text.length(); i++) {
        if (text.charAt(i) == gesucht) {
            return true;
        }
    }
    return false;
}
```

Ein Treffer genügt als Beweis, dass das Zeichen vorkommt. Deshalb ist `return true;` innerhalb der Schleife richtig. Ein einzelnes anderes Zeichen beweist noch nicht, dass der gesamte Text das gesuchte Zeichen nicht enthält.

Wenn die Schleife regulär endet, wurden alle Zeichen ohne Treffer geprüft. Dann liefert die Methode `false`. Bei einem leeren Text wird der Schleifenrumpf gar nicht ausgeführt; auch dafür ist das abschließende `return false;` notwendig. Ohne diese Rückgabe gäbe es einen möglichen Ablauf ohne Ergebnis, obwohl die Methode `boolean` verspricht.

Testaufrufe innerhalb von `main`

```
IO.println(enthaelt("Methoden", 'h')); // true
IO.println(enthaelt("Methoden", 'M')); // true
IO.println(enthaelt("Methoden", 'x')); // false
IO.println(enthaelt("", 'h'));         // false
```

Combined: Berechnungen ohne versteckten Zustand

07 Zwischenwerte lokal halten | Combined

a) Der ursprüngliche Code gibt zuerst 6 und anschließend 12 aus:

Aufruf	Startwert von `summe`	Addition	Rückgabe
Erster Aufruf mit 3	`0`	`1 + 2 + 3`	`6`
Zweiter Aufruf mit 3	`6`	`1 + 2 + 3`	`12`

Die übergeordnete Variable wird nur einmal mit `0` initialisiert. Auch ohne `= 0` erhält sie automatisch den Standardwert `0`. Nach einem Neustart sind die Ausgaben daher weiterhin `6` und `12`. Die Methode setzt die Variable nicht zurück. Nach dem ersten Aufruf bleibt `summe` bei `6`; damit beginnt der zweite Aufruf.

b) Die laufende Summe ist ein Zwischenwert dieses einen Aufrufs und gehört deshalb in die Methode:

```
int addiereBis(int grenze) {
    int summe = 0;
    for (int i = 1; i <= grenze; i++) {
        summe += i;
    }
    return summe;
}
```

Jeder Aufruf legt eine eigene lokale Variable `summe` an. `int summe = 0;` setzt sie jedes Mal auf `0`, sodass frühere Aufrufe keinen Einfluss haben.

Ohne `= 0` lässt sich diese Variante nicht kompilieren: Lokale Variablen erhalten keinen Standardwert. `summe += i` liest zuerst den bisherigen Wert von `summe` und addiert dann `i`. Bei `grenze = 0` würde auch `return summe;` einen noch nicht zugewiesenen Wert lesen. Die Zuweisung darf nach der Deklaration stehen, muss aber vor dem ersten Lesen erfolgen. Deshalb stellen wir die Initialisierung mit `0` wieder her.

c) Testprogramm zur Methode aus b):

```
void main() {
    IO.println(addiereBis(0));  // 0
    IO.println(addiereBis(1));  // 1
    IO.println(addiereBis(3));  // 6
    IO.println(addiereBis(1000)); // 500500
    IO.println(addiereBis(3));  // 6
    IO.println(addiereBis(3));  // 6
}
```

Für `grenze = 0` startet `i` mit 1. Die Bedingung `i <= grenze` ist sofort falsch; die Summe bleibt 0. Auch der größte erlaubte Wert 1000 ist unproblematisch: Die Summe beträgt 500500 und passt in `int`.

Bewusst erhaltener Zustand: Ein Spiel kann die Gesamtzahl der bisher gespielten Runden über mehrere Methodenaufrufe hinweg speichern. Dann ist gerade beabsichtigt, dass ein späterer Aufruf den bisherigen Stand kennt. Eine reine Summenberechnung benötigt diese Abhängigkeit nicht.

Combined: Drei Varianten des ggT

08 Den ggT mit Modulo berechnen | Combined

a) Die Eingaben sind positiv. Die Subtraktionsvariante endet bei gleichen Werten:

```
int ggtSubtraktion(int a, int b) {
    while (a != b) {
        if (a > b) {
            a -= b;
        } else {
            b -= a;
        }
    }
    return a;
}
```

Die Modulo-Variante speichert zuerst den Rest. Erst danach werden `a` und `b` überschrieben:

```
int ggtModulo(int a, int b) {
    while (b != 0) {
        int rest = a % b;
        a = b;
        b = rest;
    }
    return a;
}
```

b) Rekursive Modulo-Variante:

```
int ggtRekursiv(int a, int b) {
    if (b == 0) {
        return a;
    }
    return ggtRekursiv(b, a % b);
}
```

Der Abbruchfall `b == 0` liefert `a` zurück und verhindert die Division durch 0. Die Methode ist endrekursiv: Nach dem rekursiven Aufruf steht keine Berechnung mehr aus. Sie reicht nur dessen Ergebnis zurück.

Solange `b > 0` gilt, liegt `a % b` zwischen 0 und `b - 1`. Der zweite Parameter wird damit bei jedem weiteren Aufruf kleiner. Er erreicht schließlich 0.

Combined: Ergebnisse und Aufwand vergleichen

08 Den ggT mit Modulo berechnen – Fortsetzung | Combined

c) Alle drei Methoden liefern dieselben Ergebnisse:

Eingaben	Subtraktion	Modulo iterativ	Modulo rekursiv
`(84, 30)`	`6`	`6`	`6`
`(17, 13)`	`1`	`1`	`1`
`(42, 42)`	`42`	`42`	`42`
`(100, 1)`	`1`	`1`	`1`

Für `(100, 1)` benötigt die gezeigte Subtraktionsvariante **99 Schleifendurchläufe**: 100 wird bis auf 1 vermindert. Die Modulo-Variante benötigt **einen Durchlauf**: Aus `(100, 1)` wird `(1, 0)`, dann endet sie.

Der Rest fasst viele Subtraktionen zusammen. Bei 100 geteilt durch 1 ist der Rest sofort 0. Allgemein gilt für die Ganzzahldivision `a = q * b + rest`. Jeder gemeinsame Teiler von `a` und `b` teilt auch `rest = a - q * b`. Umgekehrt teilt jeder gemeinsame Teiler von `b` und `rest` auch `a`. Daher bleibt der ggT bei `(a, b)` zu `(b, rest)` erhalten.

Beispiel für den Verlauf mit Modulo

```
(84, 30) -> (30, 24) -> (24, 6) -> (6, 0)
Ergebnis: 6
```

d) Diese `main` wird zusammen mit den drei Methoden gespeichert:

```
void main() {
    int a = Integer.parseInt(IO.readln("Erste Zahl: "));
    int b = Integer.parseInt(IO.readln("Zweite Zahl: "));
    IO.println(ggtSubtraktion(a, b));
    IO.println(ggtModulo(a, b));
    IO.println(ggtRekursiv(a, b));
}
```

Bei den Eingaben 84 und 30 wird dreimal 6 ausgegeben. Die Variablen `a` und `b` in `main` bleiben nach jedem Aufruf 84 und 30. Jede ggT-Methode arbeitet mit eigenen Parameterkopien.

Einlesen und Ausgeben erfolgen ausschließlich in `main`. Dadurch lassen sich die ggT-Methoden ebenso mit festen Zahlen oder Ergebnissen anderer Berechnungen verwenden.

Challenge: Ziffern zählen und Ergebnisse zusammenführen

09 Ziffern rekursiv zählen | Challenge

a) Die Datei `K05Aufgabe09.java` enthält die nicht endrekursive Variante. Der Abbruchfall behandelt eine einzelne Ziffer einschließlich der Zahl 0:

```
void main() {
    int anzahl = zaehleZiffer(7, 707);
    IO.println(anzahl);
}

int zaehleZiffer(int ziffer, int zahl) {
    if (zahl < 10) {
        return zahl == ziffer ? 1 : 0;
    }
    int bisher = zaehleZiffer(ziffer, zahl / 10);
    return bisher + (zahl % 10 == ziffer ? 1 : 0);
}
```

Bei einer mehrstelligen Zahl zählt der rekursive Aufruf zunächst die Treffer in allen Ziffern außer der letzten. Nach seiner Rückkehr wird für die letzte Ziffer 1 oder 0 hinzugefügt. Genau diese noch ausstehende Addition macht die Variante nicht endrekursiv.

Die Bedingung `zahl < 10` verhindert, dass nach der letzten Ziffer noch eine zusätzliche führende Null gezählt wird. Gleichzeitig gilt für die ursprüngliche Eingabe 0: `zaehleZiffer(0, 0)` liefert 1, `zaehleZiffer(7, 0)` liefert 0.

Aufruf	Rückgabewert
`zaehleZiffer(7, 70701277)`	4
`zaehleZiffer(0, 70701277)`	2
`zaehleZiffer(7, 111)`	0
`zaehleZiffer(0, 0)`	1
`zaehleZiffer(7, 0)`	0
`zaehleZiffer(0, 1000)`	3
`zaehleZiffer(7, 707)`	2

Zum Testen kann der Aufruf in `main` jeweils ersetzt werden. Die beiden bedingten Ausdrücke mit `?:` liefern 1 bei einem Treffer und sonst 0; die Methode gibt eine Anzahl vom Typ `int` zurück.

Challenge: Laufzeitstack der nicht endrekursiven Variante

09 Ziffern zählen – Aufbau und Abbau | Challenge

b) Zeilennummern dienen hier nur der Erläuterung und gehören nicht in die Java-Datei:

```
 1  void main() {
 2      int anzahl = zaehleZiffer(7, 707);
 3      IO.println(anzahl);
 4  }
 5
 6  int zaehleZiffer(int ziffer, int zahl) {
 7      if (zahl < 10) {
 8          return zahl == ziffer ? 1 : 0;
 9      }
10      int bisher = zaehleZiffer(ziffer, zahl / 10);
11      return bisher + (zahl % 10 == ziffer ? 1 : 0);
12  }
```

Die Tabelle zeigt den Stack von unten nach oben, also zuerst `main`. Der letzte Eintrag ist jeweils aktiv. Kurzschreibweise: `Z(707)` bedeutet `zaehleZiffer(7, 707)`; der Parameter `ziffer` ist in allen Aufrufen 7.

Schritt und Stack	Aktive Zeile und Fortsetzung
1. `main`	Zeile 2 ruft `Z(707)` auf. Danach soll der Rückgabewert in `anzahl` gespeichert werden.
2. `main / Z(707)`	Die Bedingung in Zeile 7 ist falsch. Zeile 10 ruft `Z(70)` auf. Danach warten Zuweisung an `bisher` und die Addition `bisher + 1` in Zeile 11.
3. `main / Z(707) / Z(70)`	Die Bedingung in Zeile 7 ist falsch. Zeile 10 ruft `Z(7)` auf. Danach folgen die Zuweisung und in Zeile 11 die Addition `bisher + 0`.
4. `main / Z(707) / Z(70) / Z(7)`	Die Bedingung in Zeile 7 ist wahr. Zeile 8 liefert 1. `Z(7)` wird vom Stack genommen.
5. `main / Z(707) / Z(70)`	Fortsetzung in Zeile 10: `bisher = 1`. Zeile 11 berechnet `1 + 0 = 1`. Rückgabe 1, dann endet `Z(70)`.
6. `main / Z(707)`	Fortsetzung in Zeile 10: `bisher = 1`. Zeile 11 berechnet `1 + 1 = 2`. Rückgabe 2, dann endet `Z(707)`.
7. `main`	Zeile 2 übernimmt 2 in `anzahl`. Zeile 3 gibt 2 aus. Anschließend endet `main`.

Rückgabewerte beim Abbau: `1` → `1` → `2`. Die wartenden Aufrufe behalten ihre eigenen Parameter. Deshalb kann beim Rückweg jeder noch seine letzte Ziffer prüfen und zur erhaltenen Anzahl addieren.

Challenge: Treffer vor dem nächsten Aufruf zählen

09 Ziffern zählen – endrekursive Variante | Challenge

c) Die Datei `K05Aufgabe09Endrekursiv.java` verwendet eine überladene Hilfsmethode. Der Aufruf mit zwei Argumenten bleibt erhalten:

```
void main() {
    int anzahl = zaehleZiffer(7, 707);
    IO.println(anzahl);
}

int zaehleZiffer(int ziffer, int zahl) {
    return zaehleZiffer(ziffer, zahl, 0);
}

int zaehleZiffer(int ziffer, int zahl, int bisher) {
    int neu = bisher + (zahl % 10 == ziffer ? 1 : 0);
    if (zahl < 10) {
        return neu;
    }
    return zaehleZiffer(ziffer, zahl / 10, neu);
}
```

`bisher` enthält die Treffer aus den bereits entfernten rechten Ziffern. Die Hilfsmethode zählt die aktuelle letzte Ziffer sofort hinzu. `neu` enthält damit alle bisherigen Treffer einschließlich dieser Ziffer.

Falls nur noch eine Ziffer vorliegt, wird `neu` zurückgegeben. Sonst erhält der nächste Aufruf die verkürzte Zahl und die neue Anzahl. Nach diesem Aufruf wird ausschließlich dessen Ergebnis weitergereicht.

Sonderfall 0: `zaehleZiffer(0, 0)` startet mit `bisher = 0`. Die letzte Ziffer ist 0, deshalb wird `neu = 1`. Wegen `zahl < 10` endet der Aufruf sofort mit 1. Für `zaehleZiffer(7, 0)` entsteht dagegen `neu = 0`.

Alle Testfälle aus a) liefern auch mit dieser Variante dieselben Ergebnisse. Beide Varianten dürfen nicht gemeinsam mit denselben Methodensignaturen in einer Datei stehen; deshalb sieht das Übungsblatt zwei Dateien vor.

Parameter der Hilfsmethode für das Beispiel `(7, 707)`

Aufruf	Treffer der letzten Ziffer	`neu`
`zaehleZiffer(7, 707, 0)`	1	1
`zaehleZiffer(7, 70, 1)`	0	1
`zaehleZiffer(7, 7, 1)`	1	2

d) Speicherbedarf: Java garantiert keine Optimierung endrekursiver Aufrufe. Unser Modell zeigt daher je Aufruf einen Stackeintrag. Einschließlich `main` benötigt die nicht endrekursive Variante höchstens vier Einträge, die endrekursive mit ihrer Einstiegsmethode fünf. `IO.println` ist dabei ausgeblendet.

Challenge: Das fertige Ergebnis nur noch zurückreichen

09 Endrekursion – Laufzeitstack und Vergleich | Challenge

d) Fortsetzung: Zeilennummern zur endrekursiven Variante aus c):

```
 1  void main() {
 2      int anzahl = zaehleZiffer(7, 707);
 3      IO.println(anzahl);
 4  }
 5
 6  int zaehleZiffer(int ziffer, int zahl) {
 7      return zaehleZiffer(ziffer, zahl, 0);
 8  }
 9
10  int zaehleZiffer(int ziffer, int zahl, int bisher) {
11      int neu = bisher + (zahl % 10 == ziffer ? 1 : 0);
12      if (zahl < 10) {
13          return neu;
14      }
15      return zaehleZiffer(ziffer, zahl / 10, neu);
16  }
```

Kurzschreibweise: `E(707, 0)` bedeutet `zaehleZiffer(7, 707, 0)`. `Z(707)` ist die Einstiegsmethode mit zwei Parametern. Der letzte Stackeintrag ist aktiv.

Stack von unten nach oben	Aktive Zeile und Fortsetzung
`main`	Zeile 2 ruft die Einstiegsmethode `Z(707)` auf.
`main / Z(707)`	Zeile 7 ruft `E(707, 0)` auf. Der Aufruf wartet auf dessen Ergebnis.
`main / Z(707) / E(707, 0)`	Zeile 11: `neu = 1`. Zeile 15 ruft `E(70, 1)` auf.
`main / Z(707) / E(707, 0) / E(70, 1)`	Zeile 11: `neu = 1`. Zeile 15 ruft `E(7, 1)` auf.
`main / Z(707) / E(707, 0) / E(70, 1) / E(7, 1)`	Zeile 11: `neu = 2`. Die Bedingung in Zeile 12 ist wahr. Zeile 13 gibt 2 zurück; `E(7, 1)` endet.
`main / Z(707) / E(707, 0) / E(70, 1)`	Fortsetzung in Zeile 15: Die erhaltene 2 wird unverändert zurückgegeben. `E(70, 1)` endet.
`main / Z(707) / E(707, 0)`	Fortsetzung in Zeile 15: Auch dieser Aufruf reicht 2 unverändert zurück.
`main / Z(707)`	Fortsetzung in Zeile 7: Die Einstiegsmethode reicht 2 zurück.
`main`	Zeile 2 speichert 2 in `anzahl`; Zeile 3 gibt 2 aus.

Die Additionen erfolgen beim Aufbau; beim Abbau wird nur noch `2` → `2` → `2` → `2` weitergereicht. Beide Varianten verkürzen die Zahl mit `zahl / 10`, bis nur eine Ziffer bleibt.

Challenge: Eine IBAN in Teilaufgaben zerlegen

10 Eine deutsche IBAN berechnen | Challenge

a) Eine mögliche Zerlegung nach der deutschen Standardregel:

Methodenkopf	Aufgabe
`String ergaenzeNullen(String text, int laenge)`	Text links bis zur gewünschten Länge mit Nullen ergänzen.
`int rest97(String ziffern)`	Rest der langen Dezimalzahl schrittweise berechnen.
`String erzeugeIban(String blz, String konto)`	BBAN bilden, Prüfzahl berechnen und IBAN zusammensetzen.
`boolean hatGueltigePruefzahl(String iban)`	Die Prüfrechnung aus e) durchführen.
`void main()`	BLZ und Kontonummer einlesen und das Ergebnis ausgeben.

Die BLZ und Kontonummer bleiben Texte. So gehen vorhandene führende Nullen nicht verloren. Das feste Land `DE` muss nicht als Parameter übergeben werden.

b) Für `"1234"` beginnt die Rechnung mit `rest = 0`. Nach jeder Ziffer ergibt sich:

```
Ziffer 1: ( 0 * 10 + 1) % 97 =  1
Ziffer 2: ( 1 * 10 + 2) % 97 = 12
Ziffer 3: (12 * 10 + 3) % 97 = 26
Ziffer 4: (26 * 10 + 4) % 97 = 70
```

Der Rückgabewert ist 70, genau wie bei `1234 % 97`. Für die 24-stellige Ziffernfolge aus der Aufgabe liefert die Methode 58.

Nach `% 97` liegt `rest` zwischen 0 und 96. Die größte nächste Ziffer ist 9; der Zwischenwert wird daher höchstens `96 * 10 + 9 = 969`. Dafür reicht `int`. Die gesamte lange Zahl bleibt ein Text.

b–c) Die Methode mit dem in c) ergänzten Doc-Comment:

```
/**
 * Berechnet den Rest einer Dezimalzahl bei Division durch 97.
 * @param ziffern nicht leerer Text aus den Ziffern 0 bis 9
 * @return Rest zwischen 0 und 96
 */
int rest97(String ziffern) {
    int rest = 0;
    for (int i = 0; i < ziffern.length(); i++) {
        int z = ziffern.charAt(i) - '0';
        rest = (rest * 10 + z) % 97;
    }
    return rest;
}
```

Warum genügt der Rest? Nach den ersten drei Ziffern gilt `123 = 97 + 26`. Mit der nächsten Ziffer 4 wird daraus `1234 = 970 + 264`. Die 970 sind durch 97 teilbar und ändern den Rest nicht. Deshalb genügt es, mit `26 * 10 + 4` weiterzurechnen. Das funktioniert bei jeder weiteren Ziffer genauso.

Challenge: Kontonummer und Prüfzahl ergänzen

10 IBAN erzeugen – Implementierung | Challenge

c) Fortsetzung: Die Methode zum Auffüllen wird für die Kontonummer und die Prüfzahl verwendet:

```
String ergaenzeNullen(String text, int laenge) {
    while (text.length() < laenge) {
        text = "0" + text;
    }
    return text;
}
```

Ein Text, der bereits lang genug ist, bleibt unverändert. Bei den vorgegebenen Eingaben sind Kürzungen nicht erforderlich.

```
/**
 * Berechnet eine deutsche IBAN nach der Standardregel.
 * @param blz Bankleitzahl als genau acht Ziffern
 * @param konto Kontonummer als eine bis zehn Ziffern
 * @return IBAN mit DE, zwei Pruefziffern und der BBAN
 */
String erzeugeIban(String blz, String konto) {
    String bban = blz + ergaenzeNullen(konto, 10);
    int pruefzahl = 98 - rest97(bban + "131400");
    String prueftext = ergaenzeNullen("" + pruefzahl, 2);
    return "DE" + prueftext + bban;
}
```

`"131400"` besteht aus der Kodierung von D und E, also `13` und `14`, und dem Platzhalter `00`. Die berechnete Prüfzahl liegt zwischen 2 und 98. Mit `"" + pruefzahl` wird sie zum Text, damit beispielsweise aus 6 die Schreibweise `"06"` entstehen kann.

d) Diese `main` ergänzt die drei Hilfsmethoden zum vollständigen Programm:

```
void main() {
    String blz = IO.readln("BLZ: ");
    String konto = IO.readln("Kontonummer: ");
    String iban = erzeugeIban(blz, konto);
    IO.println("IBAN: " + iban);
}
```

Beispielrechnung für BLZ `12345678` und Kontonummer `123456`

```
Kontonummer aufgefüllt:   0000123456
BBAN:                    123456780000123456
Ziffern für Prüfrechnung: 123456780000123456131400
Rest bei Division durch 97: 58
Prüfzahl: 98 - 58 = 40
IBAN: DE40123456780000123456
```

Die Hilfsmethoden rechnen nur mit den übergebenen Werten. Die Aufforderungen zur Eingabe und die Ausgabe der IBAN stehen ausschließlich in `main`.

Challenge: Testfälle und Kontrolle der Prüfzahl

10 IBAN prüfen – Ergebnisse und Grenzen | Challenge

d) Fortsetzung: Für die BLZ `12345678` ergeben sich folgende Ergebnisse:

Kontonummer	Rest bei Erzeugung	Prüfzahl	IBAN
`123456`	`58`	`40`	`DE40123456780000123456`
`1234567890`	`11`	`87`	`DE87123456781234567890`
`5`	`92`	`06`	`DE06123456780000000005`

Der zweite Fall prüft eine bereits zehnstellige Kontonummer: Es darf keine weitere Null ergänzt werden. Der dritte Fall verlangt neun führende Nullen bei der Kontonummer und eine führende Null bei der Prüfzahl. Er deckt damit beide Anwendungen der Auffüllmethode ab.

e) Die ersten vier Zeichen werden für die Kontrolle ans Ende verschoben. Da das Land laut Vorgabe immer `DE` ist, kann seine Kodierung `1314` direkt eingesetzt werden:

```
boolean hatGueltigePruefzahl(String iban) {
    String bban = iban.substring(4);
    String pruefzahl = iban.substring(2, 4);
    String ziffern = bban + "1314" + pruefzahl;
    return rest97(ziffern) == 1;
}
```

Die Prüfzahl bleibt bei der Kontrolle erhalten. Für das erste Beispiel wird also `123456780000123456131440` geprüft; bei der Erzeugung stand an dieser Stelle noch `00`.

IBAN	Rest bei Kontrolle	Rückgabewert
`DE40123456780000123456`	`1`	`true`
`DE87123456781234567890`	`1`	`true`
`DE06123456780000000005`	`1`	`true`
`DE41123456780000123456`	`2`	`false`

Im letzten Fall wurde die Prüfzahl von 40 auf 41 erhöht. In der umgestellten Zahl erhöht das den Wert um 1; der Rest steigt von 1 auf 2. Die Prüfrechnung erkennt diese Änderung.

Zum Testen kann innerhalb von `main` ergänzt werden:

```
IO.println(hatGueltigePruefzahl(iban)); // true
IO.println(hatGueltigePruefzahl("DE41123456780000123456")); // false
```

Praxisgrenze: Das Ergebnis `true` bestätigt hier nur die Modulo-97-Prüfung für die vorausgesetzte Eingabeform. Es bestätigt weder die Existenz eines Bankkontos noch die Zuordnung zu einer Person. Bei der tatsächlichen Erzeugung können außerdem bankspezifische Regeln zur Bildung der BBAN gelten; diese Aufgabe behandelt ausschließlich die Standardregel.

Hintergrund: Deutsche Bundesbank – IBAN-Regeln